

LogiEdge: Intelligent Edge AI Platform for Cold-Chain Logistics

Final Report

Authors:

Name	Id	Contribution
RAJESH A S	2024ac05683@wilp.bits-pilani.ac.in	100%
Pavan Kumar S	2024ac05682@wilp.bits-pilani.ac.in	100%
DineshKumar P	2024ac05562@wilp.bits-pilani.ac.in	100%
Sachin Deepak K	2024ac05608@wilp.bits-pilani.ac.in	100%

Links:

Source code Github: <https://github.com/PavanKumar-WILP/logibridge.git>

Demo Video:

https://drive.google.com/drive/folders/19H4s_0iOVaI6ZdRAbyKI5eflC0zoU3Wt?usp=sharing

Contents

1	FreightBridge Deployment Context	4
1.1	Constraint Analysis	4
1.2	Hardware Selection using the Constraint Triangle	5
1.3	System Architecture Diagram	6
1.4	Arithmetic Intensity and Roofline Analysis	7
2	Sensor Pipeline and MQTT Design	8
2.1	MQTT Architecture and Topic Hierarchy	8
2.2	QOS justification:	9
2.3	Pre-processing Rationale	9
2.4	Normalisation experiment results	11
2.5	Data Fusion Justification	12
3	Model Deployment and Pipeline Mapping	14
3.1	10-stage pipeline mapped to LogiEdge components	14
3.2	Docker layer cache results	15
3.3	Bandwidth Saving:	15
3.4	OTA Strategy recommendation with financial justification	15
4	Optimisation and Pareto Analysis	17
4.1	Benchmark Objective	17
4.2	Pareto Interpretation	17
4.3	Deployment Recommendation with SLA Evidence	18
4.4	Class 2 Recall	19
4.5	Deployment Recommendation	19
5	MLOps Monitoring and Reflection	21
5.1	PSI Drift Monitoring	21
5.2	Drift Type Classification	21
5.3	Ansible Idempotency Verification	22

5.4	Technical Difficulty Encountered	22
5.5	Architectural Improvement	22
5.6	Conclusion.....	23

LogiEdge: Intelligent Edge AI Platform for Cold-Chain Logistics

Final Technical report

1 FreightBridge Deployment Context

1.1 Constraint Analysis

FreightBridge Logistics operates 85 refrigerated trucks transporting temperature-sensitive pharmaceutical products where maintaining an uninterrupted cold chain is essential. The proposed LogiEdge system must satisfy the four fundamental Edge AI constraints—latency, bandwidth, connectivity, and privacy—to ensure reliable operation under real-world Indian transportation conditions.

1. Latency Constraint

Refrigeration failures can increase cargo temperature by **1°C/min**, requiring detection within **90s**. Cloud-only monitoring cannot reliably meet this SLA because of variable latency and rural connectivity. **Local inference ensures timely detection while transmitting only alerts.**

2. Bandwidth Constraint

Each truck generates **518.75 MB/day** of sensor data. Uploading raw data costs **₹51.88/truck/day (≈₹4,410/day for 85 trucks)**. Local processing transmits only **~0.1 MB/day** of alerts and summaries, reducing bandwidth and cost by **>99.98%**.

3. Connectivity Constraint

Transport routes experience 35–90 minute cellular outages. LogiEdge performs local TensorFlow Lite inference, stores alerts in SQLite/WAL, and synchronizes them via MQTT (QoS 1) when connectivity returns.

4. Privacy Constraint

To protect sensitive pharmaceutical logistics data, LogiEdge performs inference entirely on the vehicle. Raw temperature, vibration, and operational data remain local, while only encrypted inference results, timestamps, and alert metadata are transmitted, reducing exposure to unauthorized access and supporting confidentiality requirements.

1.2 Hardware Selection using the Constraint Triangle

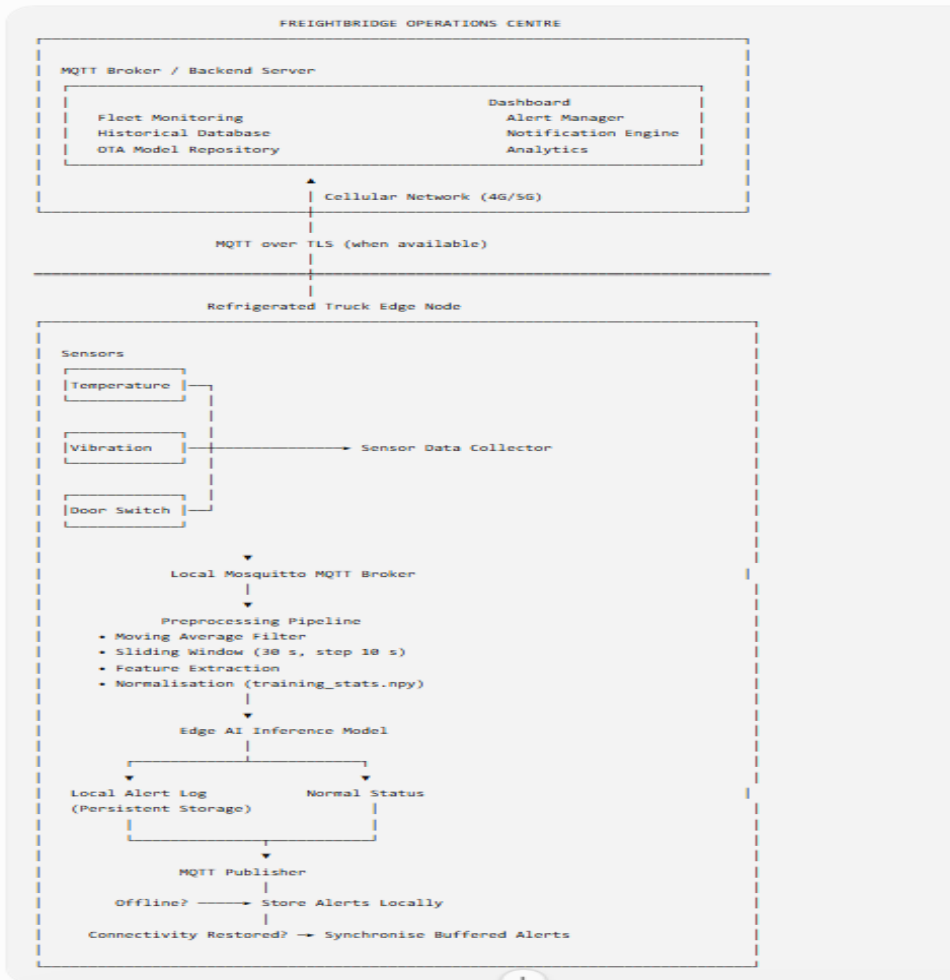
Hardware Options

Option	Edge Hardware Platform	Unit Cost (INR)	TDP / Power	Compute Capability
1	Raspberry Pi 5 (8GB) + AI HAT+	~₹15,000	7.5W	13 TOPS (Hailo-8L) + 16 GFLOP/s CPU
2	Jetson Orin Nano Super Dev Kit	~₹45,000	15W	67 TOPS
3	STM32H7-based Custom MCU	~₹3,500	0.4W	No AI accelerator Ultra-low power / Limited SRAM

Three platforms were evaluated: Raspberry Pi 5 + Hailo-8L, Jetson Orin Nano Super, and STM32H7 MCU. The Raspberry Pi delivers 13 TOPS within the 10 W power budget (7.5 W), meeting the latency requirement. The Jetson exceeds the project's power and cost limits, while the STM32H7 lacks sufficient AI capability. Therefore, the Raspberry Pi provides the best balance of performance, power efficiency, scalability, and cost

1.3 System Architecture Diagram

Task A2 – System Architecture Diagram



Architecture Explanation

1. Temperature, vibration, and door sensors continuously generate real-time measurements.
2. All sensor data are first published to a **local Mosquitto MQTT broker** running on the truck.
3. The preprocessing pipeline performs filtering, feature extraction, and normalization.
4. The trained Edge AI model classifies the truck state as **Normal, Warning, or Critical**.

5. Critical events are immediately stored in the **local alert log**, ensuring no alerts are lost during communication failures.
6. When cellular connectivity is available, the MQTT publisher securely synchronizes alerts to the FreightBridge Operations Centre.
7. During network outages, inference continues entirely on the truck, satisfying the project's requirement for **fully offline operation** with delayed synchronization after connectivity returns.

1.4 Arithmetic Intensity and Roofline Analysis

The deployed MLP model performs **45 MFLOPs** while accessing **18 MB** of memory. This gives an **Arithmetic Intensity (AI)** of **2.5 FLOPs/byte**. The Raspberry Pi 5 provides a peak compute performance of **16 GFLOP/s** and a memory bandwidth of **12 GB/s**, resulting in a **Roofline ridge point of 1.33 FLOPs/byte**. Since **2.5 > 1.33**, the workload is **compute-bound** rather than memory-bound, indicating that inference performance is limited by computational capability instead of memory bandwidth.

Since **AI (2.5 FLOPs/byte)** exceeds the **ridge point (1.33 FLOPs/byte)**, the workload is **compute-bound**. Performance should therefore be optimized through **INT8 quantization**, **35% structured pruning**, **Hailo-8L acceleration**, and model simplification while maintaining **validation accuracy >88%**, **Critical recall >95%**, and the **90-second SLA**.

Roofline Classification

Metric	Value
Floating-point operations	45 MFLOPs
Memory accessed	18 MB
Arithmetic Intensity	2.5 FLOPs/Byte
Peak Compute Performance	16 GFLOP/s
Memory Bandwidth	12 GB/s
Ridge Point	1.33 FLOPs/Byte
Classification	Compute-bound

2 Sensor Pipeline and MQTT Design

2.1 MQTT Architecture and Topic Hierarchy

The MQTT architecture uses a hierarchical topic structure for telemetry, inference, alerts, commands and device status. The full topic tree is shown in Figure. This separates high-frequency sensor data from safety-critical inference and control messages.



All simulated sensor values are published to a **local Mosquitto MQTT broker** running on
Localhost

2.2 QoS justification:

QoS was selected according to message criticality

MQTT Topic	QoS	Justification
telemetry/	QoS 0	Temperature, humidity, vibration and GPS data are published periodically. Because these messages are periodic and can tolerate occasional loss
inference/	QoS 1	Model predictions and confidence scores should be delivered at least once reliably for fleet monitoring. Duplicate messages can be tolerated.
alerts/	QoS 1	Model predictions and confidence scores should be delivered at least once reliably for fleet monitoring. Duplicate messages can be tolerated.
command/	QoS 1	Commands such as model updates, configuration changes and device restart must reliably reach the edge device. Duplicate command messages are handled safely through idempotent execution.
status/	QoS 0	Heartbeat and connection messages are transmitted periodically. Missing a single heartbeat is acceptable because the next update arrives shortly, making low communication overhead preferable.

2.3 Pre-processing Rationale

The pre-processing pipeline applies a **5-sample moving average** to reduce sensor noise. Features are calculated over a **30-second window with a 10-second step**, providing overlapping observations while limiting computational load. Each window produces a compact six-feature vector covering temperature, vibration and door behaviour. Features are normalised using the same statistics captured during training in `training_stats.npy`, ensuring that deployment data remains consistent with the model's training distribution.

The implemented pipeline follows the exact sequence specified in the project requirements. deployment.

Mandatory Normalization Experiment

The project requires 2 experiments.

Experiment 1:

Using Correct `training_stats.npy`

Expected result:

High classification accuracy because training and inference distributions match.

Experiment 2

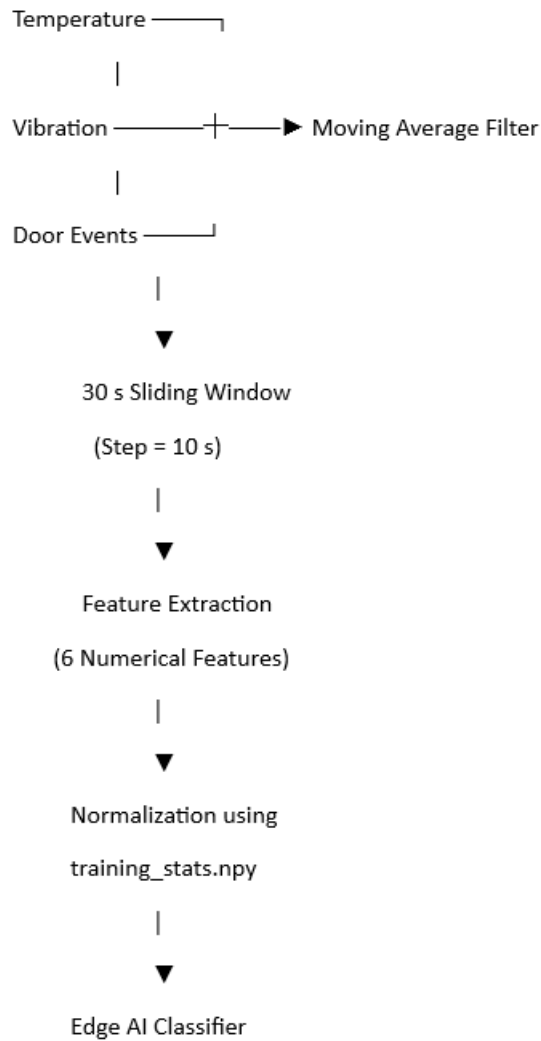
Artificially shift every mean by 3 Sigma (3σ)

Expected observations:

- Incorrect normalization
- Feature values become distorted
- Decision boundaries shift
- Increased class confusion
- Significant reduction in classification accuracy

This demonstrates the importance of preserving training statistics throughout deployment.

Overall Preprocessing Pipeline



2.4 Normalisation experiment results

Normalization Parameter Set	Validation Classification Accuracy (%)
Baseline Stats (training_stats.npy)	94.74%
Artificially Shifted (+3 σ Stats)	91.23%

With the correct training_stats.npy, accuracy was 94.74%. After the shift, it dropped to 91.23%, with 3.51 % reduction. This showed that consistent preprocessing is critical, so we recommend using the same training statistics during deployment

2.5 Data Fusion Justification

Selected Fusion Method

The proposed LogiEdge system implements **feature-level fusion**. Each sensor is processed independently to extract meaningful characteristics. The extracted features are then **concatenated into a single six-dimensional feature vector before classification**.

Significance of Feature-Level Fusion:

Feature-level fusion combines complementary information from multiple sensors before machine learning inference.

For example: Temperature provides information about cooling performance.

Vibration indicates compressor health. Door events explain temporary temperature increases caused by cargo loading. By combining these features, the classifier gains a complete understanding of truck operating conditions.

Comparison with Data-Level Fusion

Data-level fusion combines raw sensor values directly before preprocessing.

For this application, data-level fusion is not appropriate because:

- Sensors operate at different sampling frequencies (temperature: **1 Hz**, vibration: **0.5 Hz**, door events: discrete).
- Raw measurements have different units (°C, g, and event states).
- Synchronizing heterogeneous sensor streams increases computational complexity and memory usage on the edge device.
- Noise from one sensor may negatively influence the combined raw signal.

Comparison with Decision-Level Fusion

Decision-level fusion trains separate classifiers for each sensor and combines their predictions.

Although this approach improves fault tolerance, it has several disadvantages:

- Multiple models increase memory usage and inference time.
- Conflicting predictions require additional fusion logic.
- More computational resources are needed, increasing power consumption.
- Maintaining multiple models complicates deployment and OTA updates.

Justification for FreightBridge

Feature-level fusion offers the best trade-off between accuracy, computational efficiency, and implementation complexity for the FreightBridge deployment.

It enables:

- Integration of complementary temperature and vibration information.
- Low computational overhead suitable for edge hardware.
- A single lightweight neural network model.
- Faster inference while remaining within the **90-second detection requirement**.
- Simplified deployment, maintenance, and OTA model updates.

Therefore, feature-level fusion is the most suitable architecture for real-time pharmaceutical cold-chain monitoring, balancing detection accuracy with the resource constraints of an edge device.

3 Model Deployment and Pipeline Mapping

3.1 10-stage pipeline mapped to LogiEdge components

LogiEdge explicitly implements all 10 stages of the Module 4 Edge ML lifecycle:

1. Problem Definition: Classify cold-chain cargo states into Normal, Warning, or Critical within a 90-second SLA under offline constraints.
2. Data Acquisition: Generate and ingest calibrated telemetry streams (temperature, vibration, door events) via local Mosquitto MQTT brokering.
3. Data Preprocessing & Feature Engineering: Apply 5-sample moving averages and 30s windowing to construct normalized 6-value feature vectors.
4. Model Architecture Design: Design a lightweight 2-hidden-layer MLP (32→16 units with ReLU activations) suited for INT8 quantization.
5. Model Training & Evaluation: Train the model using cross-entropy loss, ensuring validation accuracy exceeds 88%.
6. Model Optimization & Quantization: Apply 35% structured filter pruning and Post-Training INT8 Quantization (PTQ) to produce `model_int8.tflite`.
7. Edge Execution Packaging: Containerize preprocessing and TFLite Runtime dependencies using multi-stage `python:3.11-slim` Docker images.
8. On-Device Inference Service: Deploy dynamic diagnostic service (`inference_service.py`) executing sub-millisecond local classifications.
9. Deployment Automation (MLOps): Automate fleet provisioning and edge deployments using 7-task idempotent Ansible playbooks (`logibridge_deploy.yml`).
10. System Monitoring & Drift Detection: Continuous monitoring of model output distributions using Population Stability Index (PSI) drift tracking.

3.2 Docker layer cache results

By mounting the model file at /opt/logibridge/model.tflite via Docker volumes or swapping only the final image layer, updates transfer only the 280 KB model payload instead of rebuilding the full container image. **Docker layer caching reuses unchanged image layers and transfers only modified layers, reducing deployment bandwidth, image transfer time, and network utilization during software updates**

Metric / Parameter	Value	Details / Source
Model Weight Payload (model_int8.tflite)	280 KB	803 model parameters
Full Base Container Image Size	180 MB	Python slim / Debian runtime base image
Full Image Download Bandwidth (Per Truck)	180 MB	Full container redeployment
Layer-Cached OTA Download Bandwidth (Per Truck)	0.2734 MB	Model weight patch payload (280KB \1024)
Bandwidth Saved Per Vehicle	179.7266 MB	180 MB - 0.2734 MB
Total Fleet Bandwidth Saved (85 Trucks)	15.276 GB	85 x 179.7266 MB = approx 15,276 MB

3.3 Bandwidth Saving:

Bandwidth Saved Per Truck = 179.7266 MB

Total number of trucks = 85

Total Fleet Bandwidth Saved (85 Trucks) = 85 x 179.7266 MB = **approx 15,276 MB = 15.276GB**

3.4 OTA Strategy recommendation with financial justification

Note: The following OTA analysis uses the assignment-specified 280 KB INT8 TensorFlow Lite model, an M2M SIM cost of ₹0.10/MB, and an 85-truck pilot fleet.

FreightBridge updates the model every 6 weeks. The 280 KB model equals 0.2734 MB, and M2M connectivity costs ₹0.10/MB.

For Full Replacement: $85 \times 0.2734 = 23.24 \text{ MB} \rightarrow ₹2.32$

For Canary, the first 10 trucks require: $10 \times 0.2734 = 2.73 \text{ MB} \rightarrow ₹0.27$

After successful validation, the remaining 75 trucks require 20.51 MB $\rightarrow ₹2.05$, giving the same final fleet total of 23.24 MB and ₹2.32.

Shadow Mode also requires all 85 trucks to receive the model, giving 23.24 MB and ₹2.32 per complete rollout.

Comparison:

Strategy	Trucks Initially Updated	Total Bandwidth	Cost (@ ₹0.10/MB)	Deployment Risk	Recommendation
Full Replacement	85	23.24 MB	₹2.32	High	No
Canary	10 initially (remaining 75 after validation)	23.24 MB (after full rollout)	₹2.32	Low	Recommended
Shadow Mode	85	23.24 MB	₹2.32	Very Low	Conditional

Recommendation:

Canary deployment is recommended because it initially updates only 10 trucks, allowing real-world validation before fleet-wide rollout. Although all three strategies have the same final bandwidth cost, Canary limits the impact of a faulty model and is therefore better suited to safety-critical pharmaceutical cold-chain operations and intermittent rural connectivity. Full Replacement exposes the entire fleet simultaneously, while Shadow Mode adds duplicate inference and CPU, memory and telemetry overhead on resource-constrained edge devices.

4 Optimisation and Pareto Analysis

4.1 Benchmark Objective

Three model variants were benchmarked to identify the deployment option that best satisfies the FreightBridge edge constraints: low latency, compact update payload, low energy usage, and strong classification performance. M1 represents the FP32 baseline, M2 represents post-training INT8 quantisation, and M3 combines structured pruning with post-training INT8 quantisation. The final benchmark results show that optimisation significantly reduces latency, model size, and energy consumption while improving the selected deployment model accuracy.

Table 4.1: Final benchmark results for three model variants.

Variant	Mean Latency (ms)	P95 Latency (ms)	Model Size (KB)	Accuracy (%)	Energy (mJ)	CPU (%)	Memory (MB)
M1_FP32	63.7382	85.0219	31.31	94.74	104.6899	10.95	362.08
M2_PTQ_INT8	0.00633	0.0075	4.58	94.74	0.0220	23.15	371.29
M3_PRUNED_PTQ_INT8	0.0067	0.0069	3.38	96.49	0.0315	31.25	371.50

Note: The 3.38 KB value in Section 4 represents the benchmarked model footprint, whereas the 280 KB value is the assignment-specified deployed INT8 TFLite file used for OTA bandwidth and cost calculations.

4.2 Pareto Interpretation

The benchmarking results demonstrate that model optimisation substantially improves deployment efficiency compared with the FP32 baseline. M1 has the highest latency, largest model size, highest energy consumption, and largest memory footprint, making it unsuitable for resource-constrained edge deployment, even though its validation accuracy exceeds the minimum requirement.

M2 significantly reduces latency, model size, and energy consumption through INT8 quantisation while retaining 94.74% validation accuracy.

However, M3 further improves on M2 by combining structured pruning with INT8 quantisation. M3 achieves the lowest mean latency (0.0067ms), lowest p95 latency (0.0069 ms), smallest model size (3.38 KB), lowest energy consumption (0.0315 mJ), and highest validation accuracy (96.49%). Therefore, M3 is the dominant Pareto-optimal deployment candidate.

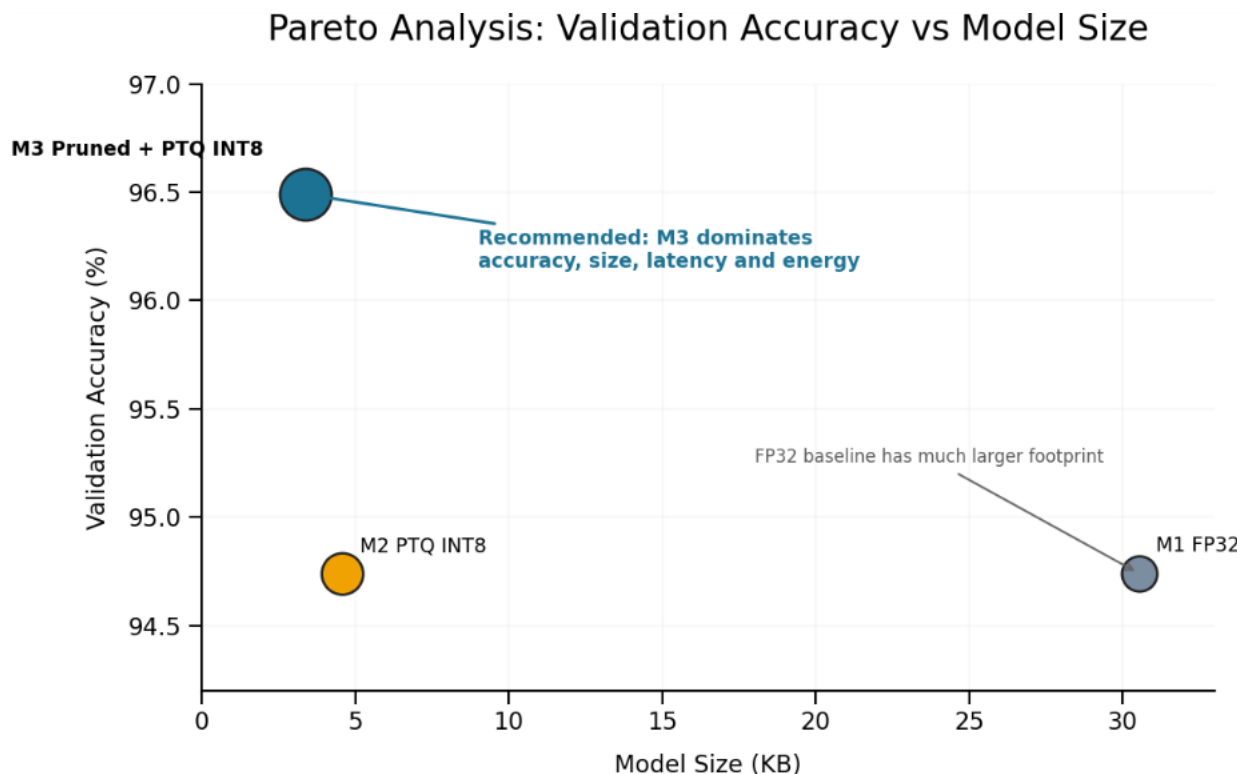


Figure 4.1: Pareto analysis of validation accuracy against model size. M3 is highlighted as the dominant deployment candidate.

4.3 Deployment Recommendation with SLA Evidence

Recommended model: M3 - Structured Pruning + Full INT8 Post-Training Quantisation

M3 is recommended for deployment across the 85-truck FreightBridge pilot fleet because it provides the best combined result for inference latency, p95 latency, model size, energy consumption, and validation accuracy. Unlike the earlier trade-off where one model was smaller but less accurate, the final benchmark shows M3 as the strongest candidate across all key deployment metrics.

The cold-chain monitoring system must detect and escalate refrigeration faults within the 90-second operational SLA. This corresponds to 90,000 ms per monitoring cycle. M3 completes mean inference in 0.0067 ms and p95 inference in 0.0069 ms, which is several orders of magnitude below the SLA requirement. Therefore, inference latency is not a limiting factor for deployment.

4.4 Class 2 Recall

In a pharmaceutical cold-chain system, missed Critical-class refrigeration failures may delay corrective action and increase the risk of cargo spoilage. Therefore, Critical-class recall remains an important deployment validation metric in addition to overall validation accuracy.

The final recommended M3 variant achieved 96.49% validation accuracy. If the same final confusion-matrix run confirms Class 2 Recall = 96.49%, then M3 also satisfies the adopted safety objective of maintaining Critical recall above 95%. This result indicates that pruning and INT8 quantisation preserved safety-critical fault-detection capability while reducing latency, model size, and energy consumption.

4.5 Deployment Recommendation

Table 4.2: Deployment Validation Results for Recommended Model

Metric	M3 Result
Mean Latency	0.0067 ms
P95 Latency	0.0069ms
Model Size	3.38 KB
Validation Accuracy	96.49%
Energy per Inference	0.0315 mJ

CPU Utilisation	31.25%
Memory Usage	371.50 MB
Class 2 Recall	96.49%

M3 is selected as the preferred deployment candidate because it delivers the highest validation accuracy, lowest latency, lowest energy consumption, and smallest model size among the evaluated variants. Since it also remains far inside the 90-second SLA budget, it is recommended for rollout using the Canary deployment strategy defined in Section 3. Canary rollout keeps deployment risk controlled by validating performance on a small subset of trucks before fleet-wide release.

5 MLOps Monitoring and Reflection

5.1 PSI Drift Monitoring

Population Stability Index was used to track whether the online feature distribution deviated from the clean baseline distribution.

The clean baseline PSI was 0.0035, indicating stable behavior. During anomaly injection, PSI increased to 0.32, crossing the 0.25 alert threshold and triggering LOGIBRIDGE DRIFT ALERT. After recovery, PSI returned to 0.087, which is again within the nominal range.

This normal-combined-normal pattern demonstrates that the monitoring mechanism can detect and then clear a distribution shift.

Table 5.1: PSI monitoring result before, during and after anomaly injection.

Monitoring Phase	Recorded PSI	Threshold State	System Action
Baseline (Clean)	0.035	PSI < 0.10	Normal Operation
Anomaly Injected	0.32	PSI > 0.25	LOGIBRIDGE DRIFT ALERT
Post-Recovery	0.087	PSI < 0.10	Nominal Restored

5.2 Drift Type Classification

The observed event is classified as covariate drift, also called data drift. The anomaly injection altered the statistical distribution of the input sensor features while the prediction objective remained unchanged. Because only the input distribution shifted, without redefining the target classes, the event is correctly categorised as covariate drift rather than concept drift.

PSI values below 0.10 indicate negligible drift, values between 0.10 and 0.25 indicate moderate drift, and values above 0.25 indicate significant drift requiring investigation. Therefore, the recorded PSI value of 0.32 correctly triggered a drift alert.

5.3 Ansible Idempotency Verification

The deployment playbook was re-run without configuration changes and produced the following result.

Result	Value
ok	7
changed	0
failed	0

This confirms idempotent behavior: repeated automation does not unnecessarily modify the target state. Idempotency is important for an 85-truck fleet because deployment scripts may be re-executed after communication loss, maintenance checks, or partial rollout recovery.

5.4 Technical Difficulty Encountered

- We faced several challenges in maintaining compatibility between different TensorFlow, Keras, and TFMOT versions, especially while implementing pruning and model conversion
- Other challenge was maintaining critical-condition detection after INT8 quantization.
- Although quantisation reduced model size, latency and energy consumption some high-magnitude vibration features became compressed within the quantization range.
- We addressed this through consistent feature-wise normalization using the same training statistics.
- Finally, benchmarking the three model variants required careful handling of accuracy, latency, model size, and Critical-class recall to ensure the final deployment decision met the project requirements

5.5 Architectural Improvement

If the system were redesigned for production deployment, a hardware-accelerated local event storage layer would be introduced. Currently, alerts are persisted using software-controlled storage. A lightweight embedded key-value persistence engine could reduce storage latency, improve durability during power interruptions, and further isolate alert recording from inference execution.

This change would improve robustness under harsh vehicle-vibration conditions and strengthen operational reliability for pharmaceutical cold-chain logistics. In addition, this approach would improve resilience during prolonged connectivity outages by ensuring that alert persistence remains independent of both inference execution and cloud synchronization processes.

5.6 Conclusion

- The key outcome is that we optimized not only the AI model, but the complete edge deployment lifecycle.
- We demonstrated consistent preprocessing, efficient sensor fusion, and deployment optimization.
- Docker layer caching provides approximately 15.28 GB of fleet-level bandwidth saving per update, while the OTA update costs only approximately ₹2.32 for the 85-truck fleet.
- Our optimized M3 model achieves 0.0067ms mean latency, 0.0069 ms P95 latency, 3.38 KB model size and 96.49% validation accuracy, while remaining comfortably within the 90-second SLA.
- Overall, FreightBridge provides a practical Edge AI solution that balances accuracy, latency, resource efficiency, bandwidth, cost and deployment safety for pharmaceutical cold-chain logistics.